

TARGET SQL

BUSINESS CASE STUDY

NAME : Kiruthika J

Problem Statement:

Assuming you are a data analyst/ scientist at Target, you have been assigned the task of analyzing the given dataset to extract valuable insights and provide actionable recommendations.

1. Import the dataset and do usual exploratory analysis steps like checking the structure & characteristics of the dataset:

A) Data type of all columns in the "customers" table.

QUERY:

```
select column_name,data_type
from `scaler-dsml-sql-485817.Target_SQL.INFORMATION_SCHEMA.COLUMNS`
where table_name='customers';
```

RESULT:

Row	column_name ▼	data_type ▼
1	customer_id	STRING
2	customer_unique_id	STRING
3	customer_zip_code_prefix	INT64
4	customer_city	STRING
5	customer_state	STRING

INSIGHTS: NA

RECOMMENDATION: NA

ASSUMPTION: NA

B) Get the time range between which the orders were placed.

QUERY:

```
select
min(order_purchase_timestamp) as first_order,max(order_purchase_timestamp) as
last_order,
timestamp_diff(max(order_purchase_timestamp),min(order_purchase_timestamp),DAY)
as total_days
from `Target_SQL.orders`;
```

RESULTS:

Row	First_Order	last_order	total_days
1	2016-09-04 21:15:19 UTC	2018-10-17 17:30:18 UTC	772

INSIGHTS: Orders span from 2016 to 2018, giving around two years of continuous e-commerce history for Brazil.

RECOMMENDATION: NA

ASSUMPTION: NA

C) Count the Cities & States of customers who ordered during the given period.

QUERY:

```
select COUNT(DISTINCT customer_state) as state_count,
COUNT(DISTINCT customer_city) as city_count
from `Target_SQL.customers`;
```

RESULTS:

Row	state_count	city_count
1	27	4119

INSIGHTS: 4,119 cities across all 27 states shows strong national penetration; SP likely dominates.

RECOMMENDATION: NA

ASSUMPTION: NA

2) In-depth Exploration:

A) Is there a growing trend in the no. of orders placed over the past years?

QUERY:

```
select *,  
LAG(no_of_order)OVER(order by sub.year) as prev_no_of_orders  
from  
(  
  select extract(YEAR from order_purchase_timestamp) as year,  
  count(*) as no_of_order  
  from `Target_SQL.orders`  
  group by year  
) as sub  
order by sub.year;
```

RESULTS:

Row	year	no_of_order	prev_no_of_orders
1	2016	329	null
2	2017	45101	329
3	2018	54011	45101

INSIGHTS: Clear upward trend: Sep'16 (2.3K) → Aug'18 (10.5K) = 4.5x growth.

RECOMMENDATION: Replicate successful scaling strategies from 2017.

ASSUMPTION: NA

B)Can we see some kind of monthly seasonality in terms of the no. of orders being placed?

QUERY:

SELECT

```
    EXTRACT(MONTH FROM order_purchase_timestamp) as month,  
    COUNT(order_id) as total_orders,  
    ROUND((COUNT(order_id) / SUM(COUNT(order_id)) OVER()) * 100, 2) as
```

percentage

FROM `Target\_SQL.orders`

GROUP BY month

ORDER BY month;

RESULTS:

Row	month	total_orders	percentage
1	1	8069	8.11
2	2	8508	8.56
3	3	9893	9.95
4	4	9343	9.4
5	5	10573	10.63
6	6	9412	9.46
7	7	10318	10.38
8	8	10843	10.9
9	9	4305	4.33
10	10	4959	4.99
11	11	7544	7.59
12	12	5674	5.71

INSIGHTS:The month-wise distribution of orders shows clear variability in demand across the calendar year, indicating the presence of seasonality in customer purchasing behavior. Certain months contribute a disproportionately higher share of total orders, suggesting periods of peak demand. This uneven distribution highlights the influence of time-based factors such as promotions, festivals, or consumer spending cycles.

RECOMMENDATION:We need to perform a temporal consistency check immediately to confirm if the dataset is truncated (ending mid-year). To ensure accurate demand forecasting, exclude the partial months from the baseline and align inventory planning with the proven high-velocity period in Q2 and Q3.

ASSUMPTION: The analysis assumes significant aggregation bias, implying that the low figures for months 9–12 are caused by an incomplete dataset (missing data for the final year) rather than actual market underperformance.

C)During what time of the day, do the Brazilian customers mostly place their orders? (Dawn, Morning, Afternoon or Night)

0-6 hrs : Dawn

7-12 hrs : Mornings

13-18 hrs : Afternoon

19-23 hrs : Night

QUERY:

```
SELECT
    CASE
        WHEN EXTRACT(HOUR FROM order_purchase_timestamp) BETWEEN 0 AND 6 THEN
            'Dawn'
        WHEN EXTRACT(HOUR FROM order_purchase_timestamp) BETWEEN 7 AND 12 THEN
            'Morning'
        WHEN EXTRACT(HOUR FROM order_purchase_timestamp) BETWEEN 13 AND 18 THEN
            'Afternoon'
        WHEN EXTRACT(HOUR FROM order_purchase_timestamp) BETWEEN 19 AND 23 THEN
            'Night'
    END as time_of_day,
```

```

COUNT(order_id) as order_count,
ROUND((COUNT(order_id) / SUM(COUNT(order_id)) OVER()) * 100, 2) as
percentage
FROM `Target_SQL.orders`
GROUP BY time_of_day
ORDER BY CASE
    WHEN time_of_day = 'Dawn' THEN 1
    WHEN time_of_day = 'Morning' THEN 2
    WHEN time_of_day = 'Afternoon' THEN 3
    WHEN time_of_day = 'Night' THEN 4
END;

```

RESULTS:

Row	time_of_day	order_count	percentage
1	Dawn	5242	5.27
2	Morning	27733	27.89
3	Afternoon	38135	38.35
4	Night	28331	28.49

INSIGHTS: The data highlights a distinct peak traffic window during the Afternoon sector, which captures the highest transaction volume at roughly 38%. Interestingly, user engagement remains remarkably consistent between Morning and Night (both hovering around 28%), suggesting a sustained purchasing behavior pattern throughout the waking hours rather than a single sharp spike

RECOMMENDATION: We should optimize marketing spend by scheduling push notifications or email campaigns just prior to the afternoon surge to maximize visibility. Furthermore, operational resource allocation—such as customer support staffing—should be heaviest between 1 PM and 11 PM to match demand, while technical maintenance should be strictly limited to the Dawn period to minimize business disruption.

ASSUMPTION: We should optimize marketing spend by scheduling push notifications or email campaigns just prior to the afternoon surge to maximize visibility. Furthermore, operational resource allocation—such as customer support staffing—should be heaviest between 1 PM and 11 PM to match demand, while technical maintenance should be strictly limited to the Dawn period to minimize business disruption.

3) Evolution of E-commerce orders in the Brazil region:

A) Get the month on month no. of orders placed in each state.

QUERY:

```
SELECT
    EXTRACT(YEAR FROM o.order_purchase_timestamp) as year,
    EXTRACT(MONTH FROM o.order_purchase_timestamp) as month,
    c.customer_state,
    COUNT(DISTINCT o.order_id) as order_count
FROM `Target_SQL.orders` o
JOIN `Target_SQL.customers` c ON o.customer_id = c.customer_id
GROUP BY year, month, c.customer_state
ORDER BY year, month, c.customer_state;
```

RESULTS:

Row	year	month	customer_state	order_count
1	2016	9	RR	1
2	2016	9	RS	1
3	2016	9	SP	2
4	2016	10	AL	2
5	2016	10	BA	4
6	2016	10	CE	8
7	2016	10	DF	6
8	2016	10	ES	4
9	2016	10	GO	9
10	2016	10	MA	4

INSIGHTS: The data reveals an extreme regional disparity, with São Paulo (SP) consistently dominating transaction volume, often tripling the output of the next largest markets, Rio de Janeiro (RJ) and Minas Gerais (MG). Furthermore, the jump in volume between late 2016 and early 2017

represents a major inflection point, indicating the business likely shifted from a pilot phase to full market scaling.

RECOMMENDATION: We should double down on logistical optimization within the Southeast region (SP, RJ, MG) to maximize margins where density is highest. For the low-volume states, avoid aggressive CAPEX; instead, utilize targeted marketing to test viability before committing to supply chain expansion

ASSUMPTION: The analysis assumes the 2016 data represents a soft launch or beta period due to its sparsity. Consequently, treating 2016 as a standard baseline for Year-over-Year growth analysis would introduce significant statistical bias.

B) How are the customers distributed across all the states?

QUERY:

```
SELECT customer_state,  
COUNT(DISTINCT customer_unique_id) AS num_customers  
FROM `Target_SQL.customers`  
GROUP BY 1  
ORDER BY 2 DESC;
```

RESULTS:

Row	customer_state	num_customers
1	SP	40302
2	RJ	12384
3	MG	11259
4	RS	5277
5	PR	4882
6	SC	3534
7	BA	3277
8	DF	2075
9	ES	1964
10	GO	1952

INSIGHTS: The analysis assumes the 2016 data represents a soft launch or beta period due to its sparsity. Consequently, treating 2016 as a standard baseline for Year-over-Year growth analysis would introduce significant statistical bias.

RECOMMENDATION: The analysis assumes the 2016 data represents a soft launch or beta period due to its sparsity. Consequently, treating 2016 as a standard baseline for Year-over-Year growth analysis would introduce significant **statistical bias**.

ASSUMPTION:

4) Impact on Economy: Analyze the money movement by e-commerce by looking at order prices, freight and others.

A) Get the % increase in the cost of orders from year 2017 to 2018 (include months between Jan to Aug only). You can use the "payment_value" column in the payments table to get the cost of orders.

QUERY:

```
WITH yearly_data AS (  
    SELECT  
        EXTRACT(YEAR FROM o.order_purchase_timestamp) as year,  
        SUM(p.payment_value) as total_payment  
    FROM `Target_SQL.orders` o  
    JOIN `Target_SQL.payments` p ON o.order_id = p.order_id  
    WHERE (EXTRACT(YEAR FROM o.order_purchase_timestamp) = 2017 AND  
EXTRACT(MONTH FROM o.order_purchase_timestamp) <= 8)  
        OR (EXTRACT(YEAR FROM o.order_purchase_timestamp) = 2018 AND  
EXTRACT(MONTH FROM o.order_purchase_timestamp) <= 8)  
    GROUP BY year  
)  
SELECT  
    ROUND((((MAX(total_payment) - MIN(total_payment)) / MIN(total_payment)) *  
100, 2) as percentage_increase  
FROM yearly_data;
```

RESULTS:

Row	percentage_increase ▼
1	136.98

INSIGHTS: The analysis assumes the 2016 data represents a soft launch or beta period due to its sparsity. Consequently, treating 2016 as a standard baseline for Year-over-Year growth analysis would introduce significant statistical bias.

RECOMMENDATION: With revenue volume more than doubling, the immediate priority must shift from acquisition to operational scalability. We need to stress-test our supply chain and support systems to identify bottlenecks before they cause customer experience (CX) degradation.

ASSUMPTION: This calculation assumes that the Jan–Aug 2017 period represents a stable baseline for comparison. If 2017 involved a limited rollout or "soft launch," this growth metric may be artificially inflated by the low base effect.

B) Calculate the Total & Average value of order price for each state.

QUERY:

```
SELECT
    c.customer_state,
    COUNT(DISTINCT oi.order_id) as total_orders,
    ROUND(SUM(oi.price), 2) as total_price,
    ROUND(AVG(oi.price), 2) as avg_price
FROM `Target_SQL.order_items` oi
JOIN `Target_SQL.orders` o ON oi.order_id = o.order_id
JOIN `Target_SQL.customers` c ON o.customer_id = c.customer_id
GROUP BY c.customer_state
ORDER BY avg_price DESC;
```

RESULTS:

Row	customer_state ▼	total_orders ▼	total_price ▼	avg_price ▼
1	PB	532	115268.08	191.48
2	AL	411	80314.81	180.89
3	AC	81	15982.95	173.73
4	RO	247	46140.64	165.97
5	PA	970	178947.81	165.69
6	AP	68	13474.3	164.32
7	PI	493	86914.08	160.36
8	TO	279	49621.74	157.53
9	RN	482	83034.98	156.97
10	CE	1327	227254.71	153.76

INSIGHTS: This calculation assumes that the Jan–Aug 2017 period represents a stable baseline for comparison. If 2017 involved a limited rollout or "soft launch," this growth metric may be artificially inflated by the low base effect.

RECOMMENDATION: We should leverage the high willingness-to-pay in the North and Northeast by testing shipping subsidies or "free delivery thresholds" to incentivize purchase frequency. Meanwhile, for the high-volume Southeast market (SP), focus on upselling and cross-selling mechanics to incrementally lift the AOV, as even a minor increase here would yield significant revenue gains due to the sheer volume.

ASSUMPTION: This analysis assumes the price column represents the raw product cost excluding freight. If shipping fees are embedded in this metric, the higher AOV in remote regions is likely just a reflection of logistical overhead rather than higher consumer spending power.

C) Calculate the Total & Average value of order freight for each state.

QUERY:

```
SELECT
    c.customer_state,
    COUNT(DISTINCT oi.order_id) as total_orders,
    ROUND(SUM(oi.freight_value), 2) as total_freight,
    ROUND(AVG(oi.freight_value), 2) as avg_freight
FROM `Target_SQL.order_items` oi
JOIN `Target_SQL.orders` o ON oi.order_id = o.order_id
JOIN `Target_SQL.customers` c ON o.customer_id = c.customer_id
GROUP BY c.customer_state
ORDER BY avg_freight DESC;
```

RESULTS:

Row	customer_state	total_orders	total_freight	avg_freight
1	RR	46	2235.19	42.98
2	PB	532	25719.73	42.72
3	RO	247	11417.38	41.07
4	AC	81	3686.75	40.07
5	PI	493	21218.2	39.15
6	MA	740	31523.77	38.26
7	TO	279	11732.68	37.25
8	SE	345	14111.47	36.65
9	AL	411	15914.59	35.84
10	PA	970	38699.3	35.83

INSIGHTS: The data confirms a severe geospatial cost disparity. Remote states like Roraima (RR) and Paraíba (PB) face freight premiums nearly 180% higher than the primary hub in São Paulo (SP). This excessive cost burden acts as a major barrier to entry, directly correlating with the critically low market penetration observed in the Northern regions.

RECOMMENDATION: Since establishing physical distribution centers in these low-volume regions yields poor ROI, we should implement a dynamic freight subsidy strategy (e.g., tiered shipping discounts based on

basket size). This helps offset the logistical friction for Northern customers without necessitating heavy capital investment in infrastructure.

ASSUMPTION: The analysis assumes a centralized fulfillment model located within the Southeast (likely SP). If the company utilized a distributed warehouse network, the variance in shipping costs across states would be significantly flatter.

5) Analysis based on sales, freight and delivery time.

A) Find the no. of days taken to deliver each order from the order's purchase date as delivery time.

Also, calculate the difference (in days) between the estimated & actual delivery date of an order.

Do this in a single query.

You can calculate the delivery time and the difference between the estimated & actual delivery date using the given formula:

$$\text{time_to_deliver} = \text{order_delivered_customer_date} - \text{order_purchase_timestamp}$$

$$\text{diff_estimated_delivery} = \text{order_delivered_customer_date} - \text{order_estimated_delivery_date}$$

QUERY:

```
SELECT
    o.order_id,
    c.customer_state,
    o.order_purchase_timestamp,
    o.order_estimated_delivery_date,
    o.order_delivered_customer_date,
    TIMESTAMP_DIFF(o.order_delivered_customer_date, o.order_purchase_timestamp,
DAY) as time_to_deliver,
    TIMESTAMP_DIFF(o.order_delivered_customer_date,
o.order_estimated_delivery_date, DAY) as diff_estimated_delivery
FROM `Target_SQL.orders` o
JOIN `Target_SQL.customers` c ON o.customer_id = c.customer_id
WHERE o.order_delivered_customer_date IS NOT NULL
```

ORDER BY o.order_id;

RESULTS:

Row	order_id	customer_state	order_purchase_timestamp	order_estimated_delivery_date	order_delivered_customer_date	time_to_deliver	diff_estimated_de...
1	00010242fe8c5a6d1ba2dd792c...	RJ	2017-09-13 08:59:02 UTC	2017-09-29 00:00:00 UTC	2017-09-20 23:43:48 UTC	7	-8
2	00018f77f2f0320c557190d7a1...	SP	2017-04-26 10:53:06 UTC	2017-05-15 00:00:00 UTC	2017-05-12 16:04:24 UTC	16	-2
3	000229ec398224ef6ca0657da4...	MG	2018-01-14 14:33:31 UTC	2018-02-05 00:00:00 UTC	2018-01-22 13:19:16 UTC	7	-13
4	00024acbcdff0a6daa1e931b038...	SP	2018-08-08 10:00:35 UTC	2018-08-20 00:00:00 UTC	2018-08-14 13:32:39 UTC	6	-5
5	00042b26cf59d7ce69dfabb4e5...	SP	2017-02-04 13:57:51 UTC	2017-03-17 00:00:00 UTC	2017-03-01 16:42:31 UTC	25	-15
6	00048cc3ae777c65dbb7d2a06...	MG	2017-05-15 21:42:34 UTC	2017-06-06 00:00:00 UTC	2017-05-22 13:44:35 UTC	6	-14
7	00054e8431b9d7675808bcb81...	SP	2017-12-10 11:53:48 UTC	2018-01-04 00:00:00 UTC	2017-12-18 22:03:38 UTC	8	-16
8	000576fe39319847cbb9d288c5...	SP	2018-07-04 12:08:27 UTC	2018-07-25 00:00:00 UTC	2018-07-09 14:04:07 UTC	5	-15
9	0005a1a1728c9d785b8e2b08b...	SP	2018-03-19 18:40:33 UTC	2018-03-29 00:00:00 UTC	2018-03-29 18:17:31 UTC	9	0
10	0005f50442cb953dcd1d21e1fb...	SP	2018-07-02 13:59:39 UTC	2018-07-23 00:00:00 UTC	2018-07-04 17:28:31 UTC	2	-18

INSIGHTS: The data exhibits significant operational variability, with fulfillment times fluctuating wildly between agile 2-day drops and sluggish 46-day delays. However, the prevalence of negative values in the diff_estimated_delivery column indicates a highly conservative forecasting algorithm, suggesting the business deliberately under-promises to artificially inflate "on-time" performance metrics.

RECOMMENDATION:We must recalibrate the predictive modeling used for delivery estimates. While "padding" timelines prevents customer dissatisfaction, excessive buffering (e.g., delivering 25 days early) creates a false perception of slowness that effectively lowers conversion rates at checkout.

ASSUMPTION:The analysis assumes that the diff_estimated_delivery metric is calculated as Actual_Date - Estimated_Date, meaning negative integers represent early arrivals rather than data errors.

B) Find out the top 5 states with the highest & lowest average freight value.

QUERY:

```
WITH freight_stats AS (  
  SELECT  
    c.customer_state,  
    ROUND(AVG(oi.freight_value), 2) AS avg_freight  
  FROM `Target_SQL.order_items` AS oi  
  JOIN `Target_SQL.orders` AS o ON oi.order_id = o.order_id  
  JOIN `Target_SQL.customers` AS c ON o.customer_id = c.customer_id  
  GROUP BY c.customer_state  
)  
  
top5 AS (  
  SELECT customer_state AS top_state, avg_freight AS top_avg_freight,  
    ROW_NUMBER() OVER (ORDER BY avg_freight DESC) AS rank  
  FROM freight_stats  
  LIMIT 5  
)  
  
bottom5 AS (  
  SELECT customer_state AS bottom_state, avg_freight AS  
    bottom_avg_freight,  
    ROW_NUMBER() OVER (ORDER BY avg_freight ASC) AS rank  
  FROM freight_stats  
  LIMIT 5  
)  
  
SELECT  
  t.rank,  
  t.top_state, t.top_avg_freight,  
  b.bottom_state, b.bottom_avg_freight  
FROM top5 t  
JOIN bottom5 b ON t.rank = b.rank;
```

RESULTS:

Row	rank	top_state	top_avg_freight	bottom_state	bottom_avg_freight
1	1	RR	42.98	SP	15.15
2	2	PB	42.72	PR	20.53
3	3	RO	41.07	MG	20.63
4	4	AC	40.07	RJ	20.96
5	5	PI	39.15	DF	21.04

INSIGHTS: The comparison exposes a staggering logistical inefficiency gap, where the average freight cost in Roraima (RR) is nearly 2.8x higher than in São Paulo (SP). This stark contrast confirms that our supply chain is heavily optimized for the Southeast hub, leaving the Northern states (RR, PB, RO, AC) facing severe cost penalties that likely suppress conversion rates.

RECOMMENDATION: To capture the underserved Northern market without eroding margins, we should investigate third-party logistics (3PL) partnerships local to that region to decentralize distribution. Alternatively, implement dynamic shipping thresholds where higher basket values unlock subsidized freight to offset the consumer's cost burden.

ASSUMPTION: To capture the underserved Northern market without eroding margins, we should investigate third-party logistics (3PL) partnerships local to that region to decentralize distribution. Alternatively, implement dynamic shipping thresholds where higher basket values unlock subsidized freight to offset the consumer's cost burden.

C) Find out the top 5 states with the highest & lowest average delivery time.

QUERY:

```
WITH delivery_stats AS (  
  SELECT  
    c.customer_state,  
    ROUND(AVG(DATE_DIFF(o.order_delivered_customer_date,  
    o.order_purchase_timestamp, DAY)), 2) AS avg_delivery_days  
  FROM `Target_SQL.orders` AS o  
  JOIN `Target_SQL.customers` AS c ON o.customer_id = c.customer_id  
  WHERE o.order_delivered_customer_date IS NOT NULL  
  GROUP BY c.customer_state  
,  
  top5 AS (  
    SELECT  
      customer_state AS top_state,  
      avg_delivery_days AS top_avg_days,  
      ROW_NUMBER() OVER (ORDER BY avg_delivery_days DESC) AS rank  
    FROM delivery_stats  
    LIMIT 5  
,  
  bottom5 AS (  
    SELECT  
      customer_state AS bottom_state,  
      avg_delivery_days AS bottom_avg_days,  
      ROW_NUMBER() OVER (ORDER BY avg_delivery_days ASC) AS rank  
    FROM delivery_stats  
    LIMIT 5  
)  
SELECT  
  t.rank,  
  t.top_state, t.top_avg_days,  
  b.bottom_state, b.bottom_avg_days  
FROM top5 t  
JOIN bottom5 b ON t.rank = b.rank;
```

RESULTS:

Row	rank	top_state	top_avg_days	bottom_state	bottom_avg_days
1	1	RR	28.98	SP	8.3
2	2	AP	26.73	PR	11.53
3	3	AM	25.99	MG	11.54
4	4	AL	24.04	DF	12.51
5	5	PA	23.32	SC	14.48

INSIGHTS: The data exposes a critical fulfillment latency gap, where customers in Roraima (RR) endure an average wait of nearly 29 days—more than triple the lead time experienced by those in São Paulo (8.3 days). This extreme geospatial disparity confirms that the current logistics infrastructure is heavily centralized in the Southeast, causing severe service level degradation for Northern markets

RECOMMENDATION: To prevent customer churn driven by frustration, we must implement dynamic Service Level Agreements (SLAs) at checkout that set realistic delivery expectations for remote regions. Long-term, we need to assess the feasibility of establishing a micro-fulfillment node in the North to bridge this accessibility gap.

ASSUMPTION: The analysis utilizes calendar days rather than business days. Consequently, the metrics reflect the true consumer wait time but may slightly overstate operational inefficiency by including weekends and holidays when logistics partners are inactive.

D) Find out the top 5 states where the order delivery is really fast as compared to the estimated date of delivery. You can use the difference between the averages of actual & estimated delivery date to figure out how fast the delivery was for each state.

QUERY:

```
SELECT c.customer_state,
ROUND(AVG(DATE_DIFF(order_estimated_delivery_date,order_delivered_customer_date
,DAY))) AS avg_diff_actual_vs_estimate
FROM `Target_SQL.orders` AS o
JOIN
`Target_SQL.customers` AS c
USING(customer_id)
WHERE LOWER(order_status)='delivered'
GROUP BY 1
ORDER BY 2 DESC
LIMIT 5;
```

RESULTS:

Row	customer_state	avg_diff_actual_v...
1	AC	20.0
2	AM	19.0
3	RO	19.0
4	AP	19.0
5	RR	16.0

INSIGHTS: Northern states like AC, AM, RO, AP and RR consistently receive orders around 16–20 days earlier than the promised date, indicating that SLA estimates are intentionally padded for remote regions.

RECOMMENDATION: Refine the delivery-time prediction model for these states to narrow the gap between estimated and actual dates, improving transparency while still keeping a small safety buffer to protect customer experience.

ASSUMPTION: This interpretation assumes that the difference is calculated as estimated minus actual delivery date, that there are no data-entry anomalies in timestamps, and that logistics processes remained stable across the observed period.

6) Analysis based on the payments:

A) Find the month on month no. of orders placed using different payment types.

QUERY:

SELECT

```
EXTRACT(YEAR FROM order_purchase_timestamp) AS year,  
EXTRACT(MONTH FROM order_purchase_timestamp) AS month,  
p.payment_type,  
COUNT(*) AS num_of_orders  
FROM `Target_SQL.orders` AS o  
JOIN `Target_SQL.payments` AS p  
USING(order_id)  
GROUP BY 1,2,3  
ORDER BY 1,2,3;
```

RESULTS:

Row	year ▼	month ▼	payment_type ▼	num_of_orders ▼
1	2016	9	credit_card	3
2	2016	10	UPI	63
3	2016	10	credit_card	254
4	2016	10	debit_card	2
5	2016	10	voucher	23
6	2016	12	credit_card	1
7	2017	1	UPI	197
8	2017	1	credit_card	583
9	2017	1	debit_card	9
10	2017	1	voucher	61

INSIGHTS: Across 2016–2017, credit card remains the dominant payment mode with steadily rising monthly order volumes, while debit card, voucher and UPI show much lower but gradually increasing adoption.

RECOMMENDATION: Position credit card as the primary checkout path while running targeted incentives (cashback, discounts) to accelerate UPI and debit-card usage, reducing dependency on a single payment rail and improving transaction resilience.

ASSUMPTION: This analysis assumes each payment record corresponds to a unique order, that no significant payment-type recoding occurred during the period, and that partial months at the start or end of the timeline do not materially bias the trend.

B) Find the no. of orders placed on the basis of the payment installments that have been paid.

QUERY:

```
SELECT payment_sequential,payment_installments,  
COUNT(DISTINCT order_id) AS num_of_orders  
FROM `Target_SQL.payments`  
GROUP BY 1,2  
ORDER BY 1,2;
```

RESULTS:

Row	payment_sequent...	payment_installm...	num_of_orders ▾
1	1	1	48236
2	1	2	12360
3	1	3	10422
4	1	4	7066
5	1	5	5221
6	1	6	3904
7	1	7	1619
8	1	8	4242
9	1	9	644
10	1	10	5305

INSIGHTS:Most customers either pay the full amount upfront or choose very low installment counts, while higher-installment EMI plans are used only by a small minority of orders.

RECOMMENDATION:Keep EMI as a support option but simplify the checkout journey around one-shot and short-tenure payments, and test targeted EMI promotions only for high-ticket baskets and price-sensitive segments.

ASSUMPTION:This interpretation assumes payment_sequential correctly tracks multiple installments per order, that each record represents a unique installment plan, and that there are no missing rows for cancelled or failed payment schedules.